

PRACTICA_03: ARQUITECTURA DE MOTORES, GAMEOBJECT Y COMPONENTES.

Nombre: Elliot Kenneth Villafaña Pasten 118006642

Usuario de Github: Piabonnie-Dev

Fecha: 17/09/2026

Objetivo y descripción general

El objetivo de esta práctica fue mejorar el código de la practica_02, para implementar una arquitectura orientado a objetos basado en componentes.

En lugar de concentrar posición, apariencia y comportamiento dentro de una sola estructura, cada entidad se representa mediante un GameObject que contiene componentes independientes.

Para el jugador se utilizaron TransformComponent, RectRenderComponent y PlayerControllerComponent. Además, se creó un objeto obstáculo estático, que se creó a partir de los componentes de transformación y renderizado únicamente.

IMPLEMENTACIÓN REALIZADA

En SDL_Applnit se ensambló el jugador por composición.

El GameObject etiquetado como "Player" recibió una posición inicial de (440, 240), escala (1, 1), un rectángulo verde de 60 × 60 píxeles y un controlador con velocidad de 300.

Posteriormente se creó un segundo GameObject llamado "Obstacle", ubicado en (150, 120), con escala 1.5 y un rectángulo rojo de 40 × 40 píxeles. El obstáculo no

incluye `PlayerControllerComponent`, por lo que permanece estático. El ciclo principal también fue simplificado. En la fase de actualización se recorren todas las entidades y se llama a `entity→Update(FIXED_TIMESTEP)`; en la fase de renderizado se recorren nuevamente y se invoca `entity→Render(renderer)`. De esta forma, el `main.cpp` ya no necesita conocer la lógica específica de cada componente.

JUSTIFICACION TEORICA:

`"std::vector<std::unique_ptr<Component>>"`

El contenedor de componentes utiliza `"std::vector<std::unique_ptr<Component>>"` porque permite almacenar objetos derivados de `Component` conservando su tipo real y su comportamiento polimorfo.

Cada `unique_ptr` posee de manera exclusiva a un componente y lo libera automáticamente cuando el `GameObject` es destruido, reduciendo el riesgo de fugas de memoria.

Si se utiliza `"std::vector<Component>"`, los objetos derivados se copiarían por valor al tamaño de la clase base. Esto produciría Object Slicing: se perderían los datos y capacidades adicionales de `TransformComponent`, `RectRenderComponent` o `PlayerControllerComponent`.

ROL DE `"dynamic_cast"` en `GetComponent<T>()`

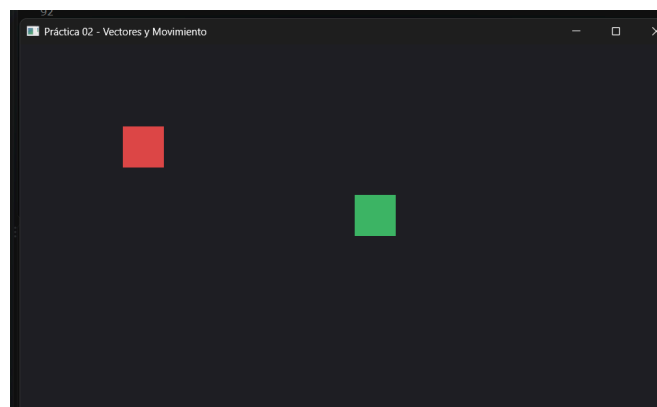
El método `GetComponent<T>()` recorre la colección interna de componentes y utiliza `dynamic_cast` para comprobar, en tiempo de ejecución, si un `Component` corresponde al tipo solicitado. Cuando encuentra una coincidencia devuelve un puntero tipado; si el `GameObject` no posee este componente, devuelve `nullptr`.

Este mecanismo permite la comunicacion entre componentes hermanos sin acoplar todo el sistema. P.e.j., RectRenderComponent obtiene el TransformComponent de su mismo GameObject para conocer la posicion y escala con las que debe dibujarse. De manera similar, PlayerControllerComponent consulta TransformComponent para modificar la posicion del jugador y puede consultar RectRenderComponent para consideras su tamaño al mantenerlo dentro de los límites de la ventana.

RESULTADO:

El resultado esperado es una ventana 960 × 540, el cual aparece un jugador verde controlable y la segunda entidad roja (el obstacle estático). Ambas son procesadas por el mismo ciclo de Update y Render, pero presentan comportamientos distintos debido a la combinación de componentes que posee cada una.

Esto demuestra el principio de composición sobre herencia y facilita que nuevas capacidades puedan añadirse sin modificar la clase GameObject.



GITHUB/REPOSITORIO: <https://github.com/Piabonnie-Dev/practica02-motor>